Pair programming

Two minds are better than one

What is pair programming?

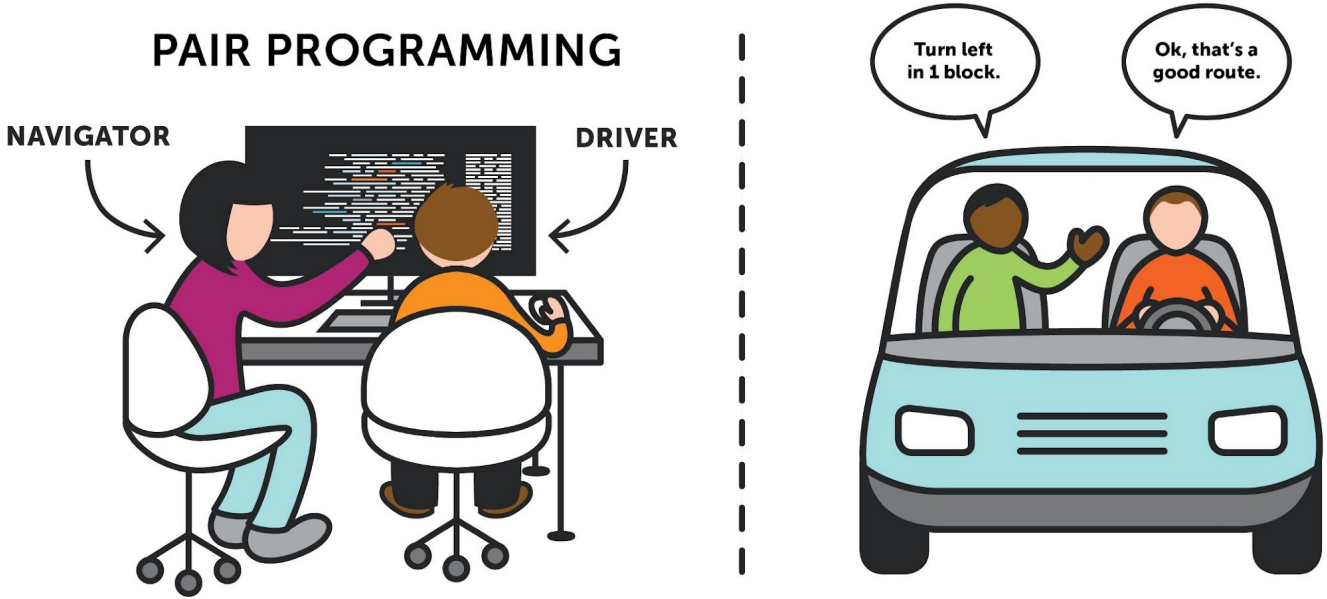
- Agile software development practice Kent **Beck**
- Two programmers working together at one computer
  - sharing one screen, keyboard, mouse

- Code is written by two people



Source: https://dzone.com/articles/extreme-programming-in-agile-a-practical-guide-for

Roles



Source: https://medium.com/@volkanbier_42259/how-to-put-pair-programming-into-action-ce9ebb9d711

Roles

- Driver
  - Writes code, focuses on syntax
- Navigator / Observer
  - Focuses on big picture
  - Is that what we actually need?
  - Is there a better way of achieving this?

⇒ Roles are **switched** constantly





Sources:
https://atypiques-mag.info/best-cliparts-2020/a-pilot-clipart-free.htm
https://www.dreamstime.com/illustration/operating-theatre.html

Benefits

- Share **knowledge**
  - Avoid knowledge silos
- Continuous code **review**
  - Early feedback → design problems
- Reduce **defects**
- Increase code **quality**
- Produce more **diverse** solutions
  - Different prior experience
- More **confident** in solutions
  - Discuss different solutions
- Onboarding new team members
- “Pair/peer pressure”
  - Focus, engagement
- Better team **communication**
- ...

Challenges

- Difficult to **schedule**
- **Experience** mismatches
- Office space
- It’s **personal**
  - Share space, expose knowledge
- Personality **conflicts**
- Exhausting

- Doubles development costs?

Empirical studies

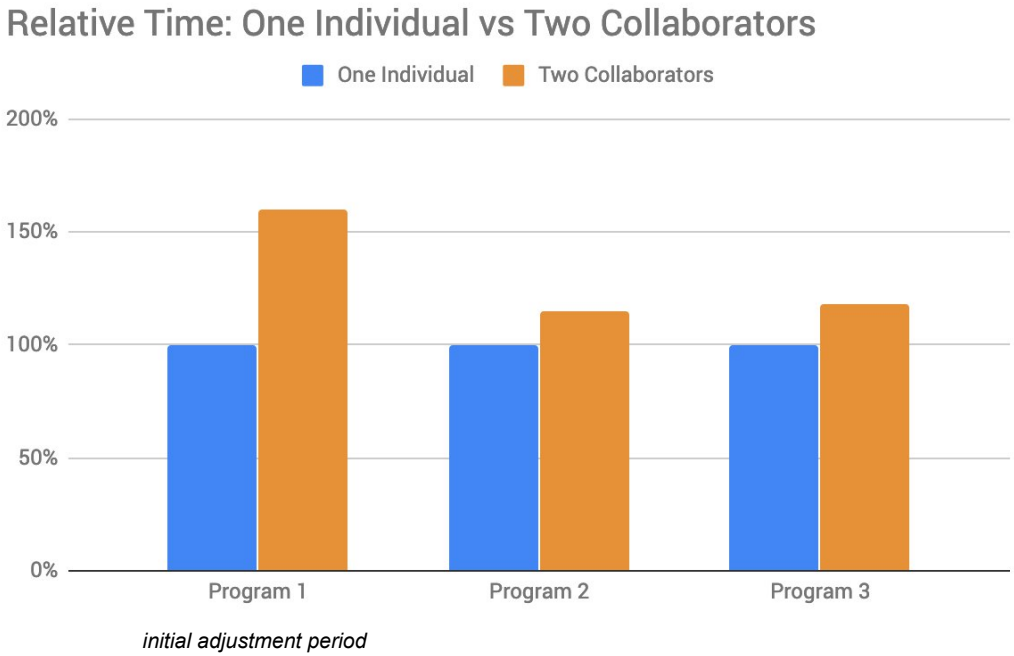
“1 + 1 = 2”

Costs: 1 + 1 = 2?

- Affordability is a key issue
- Two developers working on one program

- Does development time double?

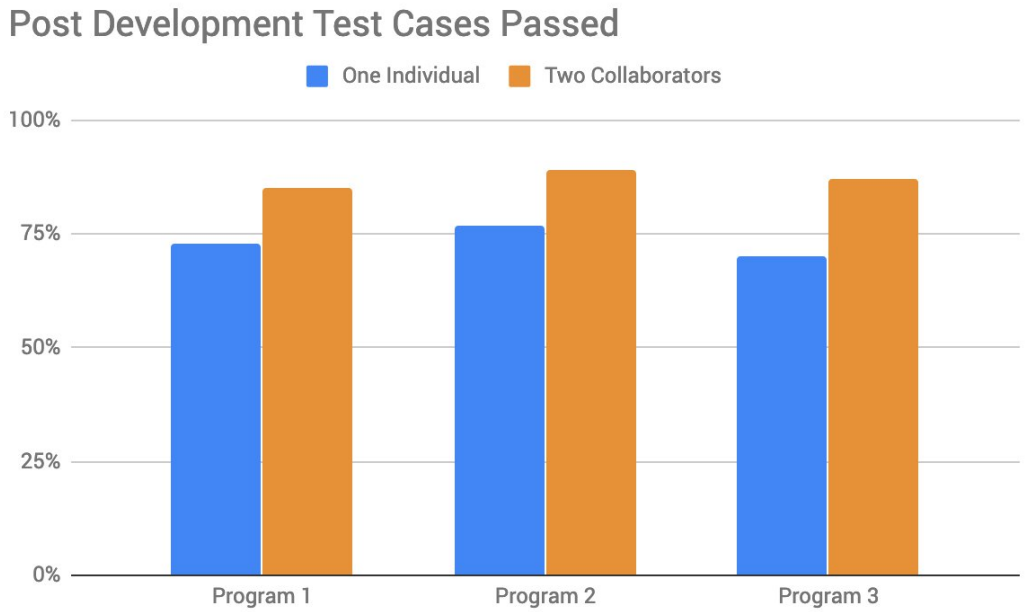
Costs: 1 + 1 = 2?



Source: The Costs and Benefits of Pair Programming. Cockburn, Alistair and Laurie Williams (2001).

- **15%** overhead
  - “1 + 1 = 1,15”

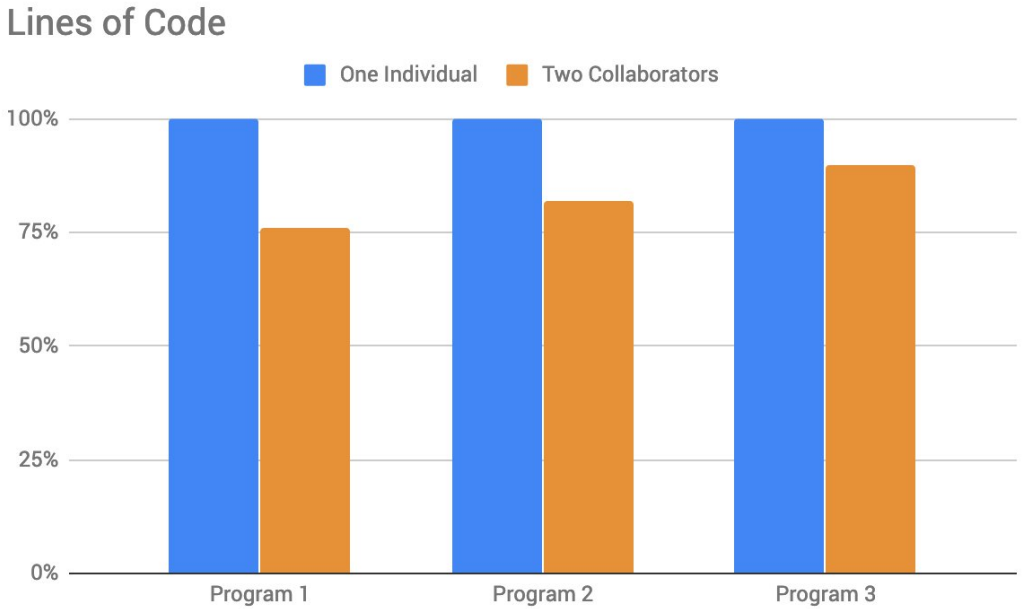
Benefits on quality: 1 + 1 > 2



- **15%** less defects

Source: The Costs and Benefits of Pair Programming. Cockburn, Alistair and Laurie Williams (2001).

Benefits on quality: 1 + 1 > 2



- Same functionality in fewer lines of code
- Better software design **quality**

Source: The Costs and Benefits of Pair Programming. Cockburn, Alistair and Laurie Williams (2001).

Pair-Programming halves the productivity of developers...

… “that would be true if the hardest part of programming was typing”

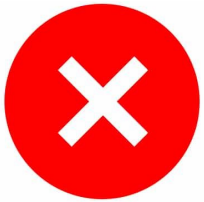
Martin Fowler

Do’s



- **Communicate** a lot → openly discuss problems
- Driver should **explain** what he/she is doing
- **Switch roles** constantly (i.e. Ping pong pair programming)
- **Switch partners** regularly (e.g. after every time)
- **Respect** your partner
- Use pairing as an opportunity to **learn** new techniques
- Expect a steep learning curve

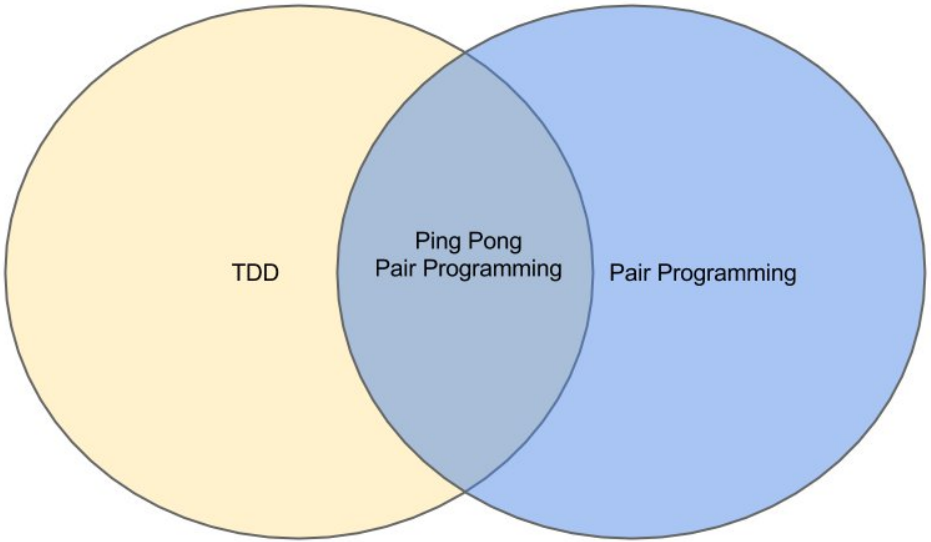
Don’ts



- Always pair with the same partner
- Suddenly grab keyboard and mouse
- Dictate your partner
- Leave partner in the middle of the session
- Be distracted or passive
- Make deconstructive comments
- Spent hours on arguing on small details (e.g. bracket placement)

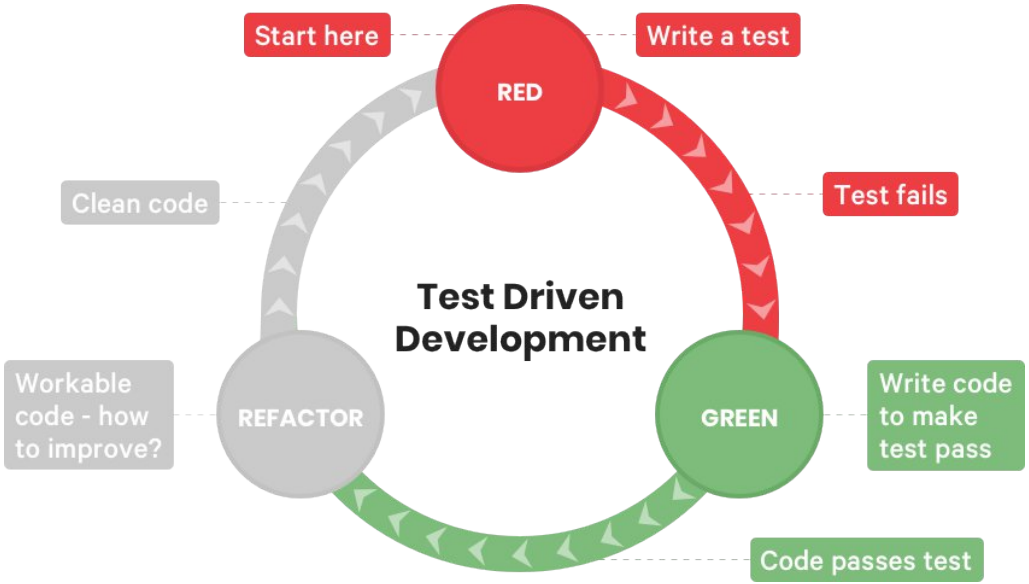
Ping pong
pair programming

Ping-Pong Pair Programming



Source: https://anthonysciamanna.com/2015/04/18/ping-pong-pair-programming.html

Recap: Red - Green - Refactor



Source: https://www.codica.com/blog/test-driven-development-benefits/

Ping-Pong Pair Programming

	A	B
Red	(1) Write a failing test	*Observer*
Green	*Observer*	(2) Make it work
Refactor	*Observer*	(3) Make it right
Red	*Observer*	(4) Write a failing test
Green	(5) Make it work	*Observer*
Refactor	(6) Make it right	*Observer*

Remote

(ping pong)

pair programming

Remote pair programming

- Similar to co-located pair programming
  - Each developer is working from a different computer

- Using software tools
  - Talk to each other and share screen

Remote pair programming

1. Partner connect using software tool
2. Driver shares screen
3. Observer watches and gives recommendations

1. Switch roles
    - Remote control partner’s computer
    - **Better: switch** screen sharing **direction**

Remote pair programming tools



- Recommendation: Discord
  - Screen sharing
  - Voice and text communication
  - Free: unlimited 1:1 meetings
  - Available for Linux, Windows and Mac
  - Allows to change screen sharing direction
    - For switching driver and navigator roles

Remote PP on public repositories



1. Driver writes code
    - Commits and pushes changes to feature branch of his/her fork
        - `git push origin` *`FEATURE_BRANCH`*

Remote PP on public repositories



2. Screen sharing direction is switched
    - New driver pulls changes from partner’s fork while being on feature branch of his/her fork
        - `git checkout -b FEATURE_BRANCH`
        - `git remote add joe[partner] git@github.com:[handle of partner]/[group].git`
        - `git pull joe FEATURE_BRANCH`

Remote PP on public repositories



3. Driver writes code
    - Commits and pushes changes to feature branch of his/her fork
        - `git push origin FEATURE_BRANCH`

3. Driver opens up a pull request

Commit message guideline for PP

- *[LF1,LF2] Commit message*
- Initials of the developers from pair programming
- last name initial followed by first name initial
- in alphabetical order
    - e.g. Patrick RATSCHILLER and Wolfgang SLANY: **[RP,SW]**

Further readings

- *Extreme Programming Explained: Embrace Change.* Kent Beck, 1999.
- *Pair Programming Illuminated.* Williams, Laurie and Robert Kessler, 2002.

Questions?